

Actividad 1. Exploración de Librerías con Meta AI.

Prompt 1. ¿Cuáles son las 5 librerías de Python más importantes para un Científico de Datos y cuál es el rol principal de cada una dentro de un flujo de trabajo?

Las 5 librerías de Python que todo Científico de Datos debe dominar, con su rol en el flujo típico de trabajo:

Las 5 esenciales

Librería	Rol principal en el flujo de trabajo	¿Para qué se usa exactamente?
Pandas	Manipulación y limpieza de datos	Es el Excel de Python. Cargas CSVs, limpias valores nulos, filtras filas, haces joins, agregaciones y transformas los datos. Es el 80% del trabajo diario. Todo empieza aquí.
NumPy	Computación numérica base	Maneja arrays y matrices multidimensionales súper rápido. Es el motor matemático debajo de casi todas las demás librerías. Operaciones vectorizadas, álgebra lineal, números aleatorios.
Matplotlib + Seaborn	Visualización y análisis exploratorio	Matplotlib es la base para cualquier gráfico: líneas, barras, scatter. Seaborn va encima y hace gráficos estadísticos bonitos con 1 línea: heatmaps, boxplots, distribuciones. Usas ambas para entender tus datos y comunicar hallazgos.
Scikit-learn	Modelado de Machine Learning	Aquí entrenas los modelos. Tiene todo: regresión lineal, random forest, K-means, SVM, validación cruzada, métricas, preprocesamiento. Es la navaja suiza para ML clásico.
Jupyter	Entorno de experimentación	No es una librería de análisis per se, pero es donde ocurre todo. Te deja mezclar código, gráficos y texto en notebooks. Ideal para iterar

Librería	Rol principal en el flujo de trabajo	¿Para qué se usa exactamente?
		rápido, documentar tu proceso y presentar resultados.

Cómo encajan en el flujo de trabajo típico

1. **Jupyter:** Abres tu notebook y empiezas a experimentar.
2. **Pandas:** Cargas tu dataset `df = pd.read_csv()` y limpias los datos.
3. **NumPy:** Pandas lo usa por debajo, pero tú lo usas directo para cálculos matemáticos o crear features.
4. **Matplotlib/Seaborn:** Haces EDA — `sns.heatmap(df.corr())`, `df.hist()` — para entender patrones y outliers.
5. **Scikit-learn:** Tomas tu `X, y` ya limpios, haces `train_test_split`, entrenas `model.fit()` y evalúas con `metrics`.

Mención honorífica: Si trabajas con deep learning, agrega **PyTorch** o **TensorFlow**. Si haces dashboards, **Plotly** o **Streamlit**.

Prompt 2. Explica detalladamente por qué es más eficiente utilizar arreglos de NumPy en lugar de listas nativas de Python cuando trabajamos con millones de datos. Menciona el concepto de 'vectorización'.

Sí, para millones de datos NumPy barre el piso con las listas de Python. La diferencia no es 10% o 20% — es de 10x a 100x más rápido y usa mucha menos memoria.

1. Cómo se guardan en memoria

Listas de Python

- Son arreglos de punteros a objetos. Cada número es un objeto Python completo con su tipo, contador de referencias, etc.
- Un entero de Python ocupa 28 bytes en 64-bit. Para 1 millón de enteros: `~28 MB` solo en los objetos, + `~8 MB` del arreglo de punteros = `~36 MB`

- Los datos están dispersos por toda la RAM. La CPU tiene que "saltar" de un lado a otro = cache misses.

Arreglos de NumPy

- Es un bloque contiguo de memoria con puros datos crudos, tipo C.
Un `int64` ocupa exactamente 8 bytes.
- Para 1 millón de `int64`: exactamente `8 MB`. Sin overhead por objeto.
- Al estar contiguos, la CPU carga bloques enteros a caché L1/L2/L3 de un jalón = cache hits masivos.

Aspecto	Lista Python	NumPy array <code>int64</code>	Diferencia con 1M datos
Memoria por elemento	~28 bytes + 8 puntero	8 bytes	4.5x menos RAM
Layout en memoria	Disperso, objetos separados	Bloque contiguo	Acceso secuencial vs saltos
Tipo de dato	Dinámico, chequeo en runtime	Fijo, conocido	Sin overhead de tipo

2. Cómo se ejecutan las operaciones

Listas de Python: bucles a nivel Python

Python

```
a = [1, 2, 3,...] # 1 millónb = [4, 5, 6,...] # 1 millónc = []for i in range(len(a)): # Este for lo ejecuta el intérprete de Python    c.append(a[i] + b[i])
```

Cada iteración hace:

1. Interpretar bytecode de Python
2. Verificar tipos de `a[i]` y `b[i]`
3. Hacer unboxing del objeto Python a int de C
4. Sumar
5. Crear nuevo objeto Python para el resultado
6. Hacer boxing de vuelta

Eso 1 millón de veces. Lento.

Arreglos de NumPy: vectorización

Python

```
a = np.array([1, 2, 3,...]) # 1 millónb = np.array([4, 5, 6,...]) # 1 millónc = a + b # No hay for explícito en Python
```

Aquí es donde entra el concepto clave de **vectorización**:

Vectorización significa que en lugar de procesar elemento por elemento con un bucle de Python, le pasas todo el arreglo a código compilado en C/Fortran que opera sobre bloques completos de memoria usando instrucciones SIMD del CPU.

Lo que pasa internamente con `a + b`:

1. Python solo hace 1 llamada: "NumPy, suma estos dos arreglos"
2. NumPy en C toma los dos bloques contiguos de memoria
3. Usa instrucciones vectorizadas AVX/SSE: suma 4 u 8 `int64` en un solo ciclo de CPU
4. No crea objetos Python intermedios
5. Devuelve un nuevo bloque contiguo

Resultado: el bucle corre en C a velocidad nativa, no en Python interpretado.

3. Benchmark real

Para sumar 10 millones de floats:

- **Lista Python:** `~1.2 s`
- **NumPy:** `~0.015 s`

Eso es **80x más rápido**. Y si usas `np.sqrt`, `np.sin`, `np.dot`, la diferencia crece porque esas funciones usan BLAS/LAPACK, librerías optimizadas durante décadas.

4. Otras ventajas de NumPy que importan con datasets grandes

- **Broadcasting:** `a * 5` multiplica todo el arreglo por 5 sin bucle. Con listas tendrías que hacer `[x*5 for x in a]`
- **Vistas sin copia:** `b = a[1000:2000]` no copia memoria, solo crea una vista. Con listas, hacer slicing copia todo.

- **Operaciones universales ufuncs:** `np.log(a)`, `np.exp(a)` están vectorizadas. Con listas usas `math.log` en un bucle.
- **Menos errores:** Al forzar un tipo único, evitas mezclar `int` con `str` por accidente en 1M de datos.

5. Cuándo NO usar NumPy

Si tu lista tiene tipos mezclados `[1, "hola", 3.14, objeto_custom]`, o haces muchos `append/pop` en medio, o tienes menos de 10 mil elementos, una lista está bien. NumPy brilla cuando tienes datos homogéneos y quieres hacer matemáticas sobre ellos.

Regla práctica: Si vas a hacer `for` sobre más de 50k elementos numéricos, NumPy casi siempre gana.

Prompt 3. Proporciona 3 ejemplos de funciones de NumPy que sean esenciales para realizar cálculos estadísticos básicos (como media o desviación estándar) y muestra cómo se aplicarían a un arreglo de una dimensión.

Claro, acá van 3 funciones de NumPy que uso siempre para estadística básica. Todas trabajan directo con arreglos de 1 dimensión:

1. `np.mean`: Calcula la media/promedio

Suma todos los valores y divide entre la cantidad de elementos.

Python

```
import numpy as np
datos = np.array([10, 15, 12, 18, 20, 14, 16])
promedio = np.mean(datos)
print(promedio) # Resultado: 15.0
```

1 línea oculta

2. `np.std`: Calcula la desviación estándar

Mide qué tan dispersos están los datos respecto a la media. Por defecto usa N en el denominador, o sea la desviación estándar poblacional.

Python

```
datos = np.array([10, 15, 12, 18, 20, 14, 16])
desv_estandar = np.std(datos)
print(desv_estandar) # Resultado: 3.162...
```

Tip: Si tus datos son una muestra y no toda la población, usa `ddof=1` para que divida entre N-1: `np.std(datos, ddof=1)`

3. np.median: Calcula la mediana

Es el valor que queda justo en medio cuando ordenas los datos. Útil porque no le afectan los valores extremos como a la media.

Python

```
datos = np.array([10, 15, 12, 18, 20, 14, 16])
mediana = np.median(datos)
print(mediana) # Resultado: 15.0
```

Con esos datos ordenados `[10, 12, 14, 15, 16, 18, 20]`, el 15 queda en el centro.

Bonus que te sirve: `np.min`, `np.max`, `np.sum` y `np.var` para mínimo, máximo, suma y varianza. Con esas 3 de arriba + estas ya cubres el 90% de la estadística descriptiva básica.

Reflexión.

Es de vital importancia tener en cuenta las herramientas mas usadas en cualquier área del conocimiento, particularmente en ciencias de la computación y programación pues su uso puede facilitar drásticamente el desarrollo y ejecución del código. Sin embargo eso no es lo único importante en nuestro desarrollo de proyectos pues el conocimiento del correcto uso de los recursos computacionales (hardware) del sistema puede ayudar a aprovechar al máximo las capacidades de este, y ahorra tiempo, recursos y salud.